

Contents

Analysis stage

Client

Client Name: Jack Moores

Occupation: teacher

Mr Moores helps run an after school computing club that encourages students to learn coding in an enjoyable way. The purpose of this project is to create a simple game that can be used as a fun and educational example for students learning object-oriented programming. The game will provide students with accessible source code that shows concepts such as classes, constructors, inheritance and encapsulation etc, in a functioning program.

The gameplay will also need to be engaging and easy to understand so that students remain interested while exploring how the code works. The gameplay will intentionally remain relatively simple so that it does not overly distract from the main purpose of the project, which is to support the teaching and learning of programming.

Project overview

My initial idea for my game will be a platformer game that takes inspiration from the game 'OFF' by Moris Ghost, 'Mario' by Nintendo and 'hollow knight' by team cherry. The game will be designed to give an immersive experience for the students, to teach them how to code game mechanic with OOP in greenfoot using java, while remaining simple and accessible enough for students that aren't big gamers.

One of my main goals of this project is to create a game that is informational and entertaining, that does not require large amounts of computer memory nor processing power and that can be played offline. By doing that it ensures that the students can still use and experience the game to its full capacity with a less powerful computer e.g. a laptop and can be played on the go.

Main issues:

The main issue the students and teacher described with existing games was that the source code is usually hidden. This makes it difficult for students to see how the game works and learn how similar features could be programmed themselves. Existing games can also include complicated features that are not useful for learning and can distract students from the code.

My game aims to avoid these problems by creating a simple platformer game using object-oriented programming. It will use familiar platforming mechanics such as movement, jumping and combat while keeping the overall design simple. The source code will be available for students to read and edit with plenty of comments to help show them what each different parts of the code do so they can see how classes, inheritance, encapsulation and polymorphism are used in a working game.

I believe my project is computationally complex because I plan to use object-oriented programming, programming in Greenfoot (java) will help to achieve these goals in creating the game as it is one of the most used programming languages. It is used for lots of games e.g. Minecraft and RuneScape and consistently ranks among the top three languages in rankings by TIOBE and RedMonk where they estimated that 39.5% of recruiters actively seek Java developers. This makes it perfect for the students as they can take the knowledge they gain from this to further their careers down the line

TIOBE ranking (22/06/26, 17:31): <https://www.tiobe.com/tiobe-index/>

RedMonk ranking (22/06/26, 17:48): <https://redmonk.com/sograzy/2026/04/14/language-rankings-1-26/>

How the problem can be solved using computational methods

Since the goal of this video game is to give students an example of how a game can be designed and programmed by giving them access to the code to explore the game's structure, understand the programming techniques used and gain broad insight into the programming process. It is necessary that I use a computer to develop the game.

Abstraction:

Abstraction is when you ignore non important details in order to focus on the key aspects of the problem. In my game I will use abstraction will help simplify mechanics to ensure the game remains easy to learn to the students which allows players to focus on the code of the game rather than being distracted.

Decomposition:

Decomposition is breaking a problem down into manageable sub-problems.

To develop my game well I want to add:

- Make a main menu
- Player movement and interactions
- Combat and encounter sequences
- Save and load functionality

The reason each component of the game will be developed as an individual module is because it makes it easier to debug, reuse code, test code and add is the overall style of oop. Once these elements are fully functional, they can be integrated into the final product and all be tested together using methods like integration testing.

Visualisation:

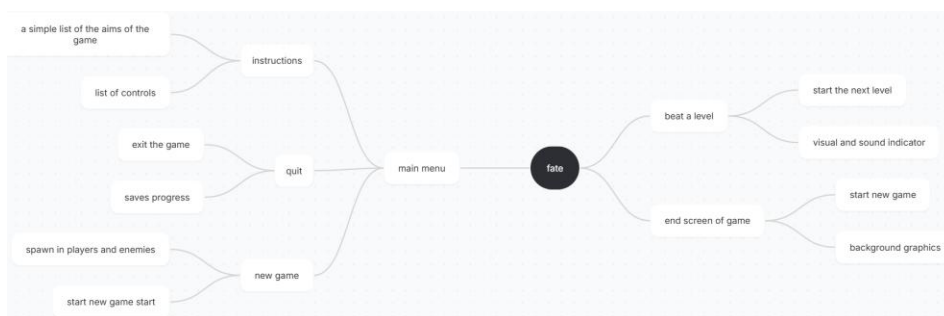
Visualisation involves imagining the final product and using diagrams to represent data and processes clearly.

Application in my game:

Throughout development I will use several visualisation methods to plan and communicate my ideas effectively, using mind maps to work out the overall game structure and identify key systems such as the menu and level transitions.

I will also use structure diagrams and flow charts to show the sequence of actions and logic used in the game e.g. how a player moves in combat and the world, layout sketches and pictures online to plan the appearance of enemies, levels and interfaces.

Visualisation helps ensure that each element of my game is well thought out before implementation, decreasing the chance of confusion or rework later in the development process.



Algorithmic thinking:

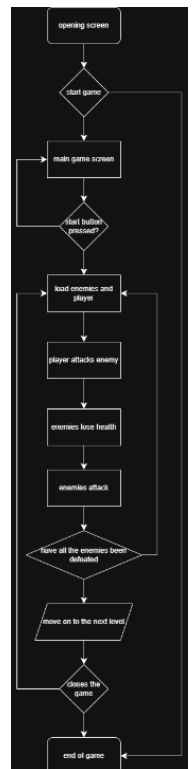
Algorithmic thinking is when you see a problem as a sequence of logical steps or processes that lead to its solution.

Application in my game:

I will use procedural thinking to design the order in which the games systems operate e.g. when the player interacts with an object, the game must first check for collision then trigger the event like a cutscene or battle sequence and finally update the game state. By defining these steps clearly through algorithms, I can ensure the game runs smoothly and efficiently.

I have created a flow chart to map out these processes which help me identify the correct order of the game and potential links between systems aka player hp and enemy damage.

This slow and step by step approach will guide my coding structure making it easier to debug and refine each part of my game.



Thinking ahead:

Thinking ahead involves anticipating possible inputs, outputs and potential errors before they occur.

Application in my game:

When developing my game, I will consider how players might interact with the game in unexpected ways e.g. by entering invalid commands, pressing keys out of order or closing the game suddenly. To handle these scenarios, I will implement error handling systems that prevent crashes and guide players back to the intended state.

I will also plan for the code to be modular for future improvement so I can add new levels later by expanding on the games concept or by adding dialogue options/features later without needing to rewrite large parts of the program. This forward-thinking approach will help ensure the game remains adaptable and maintainable.

Stakeholders

Stakeholders are individuals or groups who have an interest in the development, quality or outcome of a project. My stakeholders and their thoughts are essential to making sure that my product is successful and meets the needs of my audience. Their influence and feedback will guide the design decisions and help define the success criteria for my final product.

The stakeholders for my game are those who may play, evaluate or make use of the game in some way.

My stakeholders:

My client Mr Moores and has students who will be the main users of the game. Because my client is also part of the stakeholder group his expectations helped shape the requirements for my game. He emphasises that the game must be accessible, simple and appropriate for classroom use, so that the game can be played easily by the students and used during lessons for teaching.

To meet these needs the game must:

- Work on school computers
- Have clear and readable text
- Have simple controls
- Be engaging and have inciteful code to learn from

And the game could have but not necessary:

- A visually enticing art style
- A story for the character

These qualities will ensure that my game supports the needs of all my key stakeholder groups.

Parents:

Parents play a role in in my game by approving the types of games their children will play as the game will be sent home to have the students study it for their homework's and own projects further. For my game to be recommended or allowed at home the game should:

- Contain no inappropriate elements
- Require no supervision

Product proposal:

To ensure that my game meets the requirements of its stakeholders I have used several different methods of research and to gather feedback. This research will focus on the users' experiences with existing platformer games as well as identifying which features are the most important to include in a new game project. The results of this research will directly influence the design of my game ensuring that it aligns with both users' expectations and clients' requirements.

Research

For my game I looked at some similar examples to get some ideas as to how and what I can implement into my game

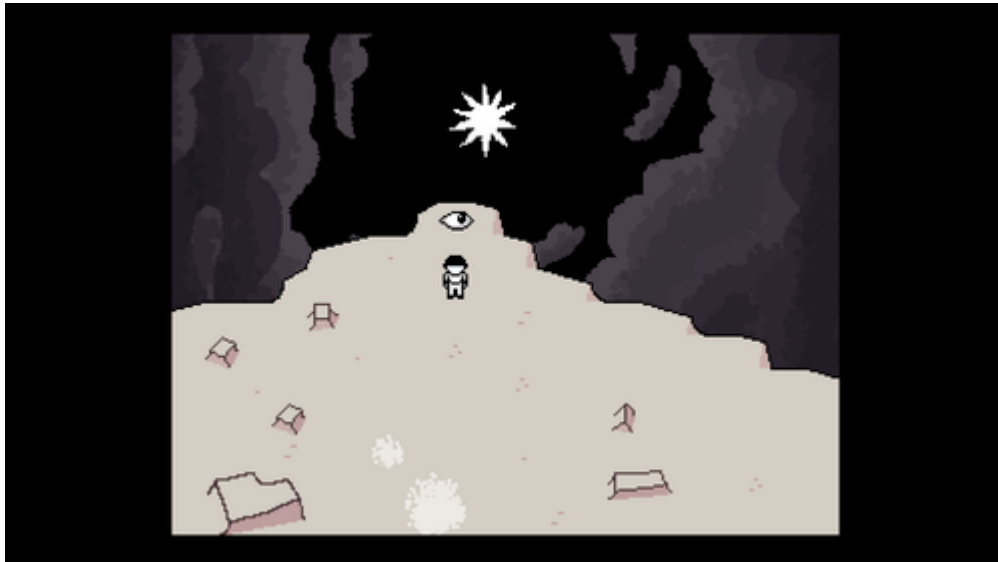


https://www.google.com/imgres?q=hollow%20knight%20gameplay&imgurl=https%3A%2F%2Fsm.ign.com%2F%2Fign_ap%2Fvideo%2Fh%2Fhollow-knight-gameplay-demonstration%2Fhollow-knight-gameplay-demonstration_1ykb.1280.jpg&imgrefurl=https%3A%2F%2Fsea.ign.com%2Fhollow-knight%2F84931%2Fhollow-knight-gameplay-demonstration&docid=9a3bTkkZDVfrqM&tbid=4_-6ugZmxfab1M&vet=12ahUKewix95LLIOaUAxSxRkEAHXT-AWkQnPAOegQIKRAB..i&w=1280&h=720&hcb=2&ved=2ahUKewix95LLIOaUAxSxRkEAHXT-AWkQnPAOegQIKRAB

14:36, 01/06/2026

Features I will/might use	justification	Features I won't use	justification
I will have the wall holding and wall jumping mechanic from hollow knight into my game	As my game will be a 2D platformer with floating islands, there needs to be a way to travers between them	Guidance system	The game I plan to make will be simple so a guidance system other than a small controls and instructions page in not needed
The health system	I like the health system in Hollow	A map	My game will not be very big, so a

	knight as it is easy to integrate into any style of game and intuitive		map is not necessary
--	--	--	----------------------



<https://imgs.search.brave.com/l92kv58R6TsW734St6WhCpgwKMeBCPpp7gQl7t2TNUI/rs:fit:500:0:1:0/g:ce/aHR0cHM6Ly9vZmZ0/aGVycGcuY29tL2lt/YWdlcy9zY3JlZW5z/aG90LTgtZW4ucG5n>

14:39, 01/06/2026

Features I will/might use	justification	Features I won't use	justification
Simple character designs	Simple characters will be easier for to create and animate so I can focus on the code while still giving my game a nice visual style	Turn based combat	My game will use real time movement and combat so turn based combat would not work with the type of game I am making
Different backgrounds for different areas	Changing the background between levels can make each level feel different without	Large amounts of dialogue	My game will focus mainly on combat so long dialog parts are not needed

	changing the overall gameplay		
--	-------------------------------	--	--



https://www.google.com/imgres?q=mario%20gameplay&imgurl=https%3A%2F%2Fytimg.com%2Fvi%2FEqb8x8CDikw%2Fhq720.jpg%3Fsqp%3D-oaymwEhCK4FEIIDSfryg4qpAxMIARUAAAAAGAEIAADIQjQAgKJD%26rs%3DAOn4CLCHKLulCkmCwj2GTOPMOZKzKAW9mA&imgrefurl=https%3A%2F%2Fwww.youtube.com%2Fwatch%3Fv%3DEqb8x8CDikw&docid=qUnuK-Jo04N3IM&tbnid=PttFegNOOK_hQM&vet=12ahUKEwiSuJWcleaUAxXKVKkEAHdyOGrAQnPAOegQIFhAB..i&w=686&h=386&hcb=2&ved=2ahUKEwiSuJWcleaUAxXKVKkEAHdyOGrAQnPAOegQIFhAB

14:39, 01/06/2026

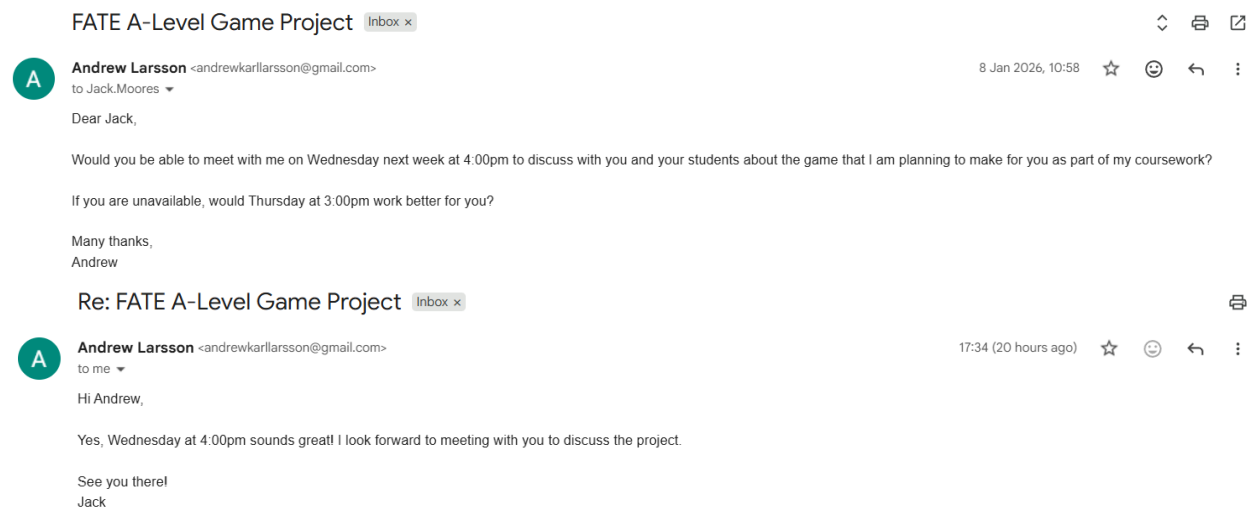
Features I will/might use	justification	Features I won't use	justification
I will use the concept of the floating platforms in my game to have enemies spawn and have the player move between them and defeat them	The game is well known and the idea to have islands is simple yet very impactful to how the gameplay can change and be expanded upon	Power up blocks	I won't have a 'levelling up' system in my game so a block to facilitate the levelling up is not needed
Level progression	Having different levels to my game will add a lot of depth while also adding some progression	Collectable items	My game does not use collectables, so coins or similar items are not needed

Interview:

I contacted my client Mr Moores to arrange an interview with him and his students in order to gain a clearer understanding of his expectations for the project and the needs of the

students. We agreed on a date to meet and discuss how my game could be used effectively within a school environment.

Below is evidence of our communication to arrange the interview.



This meeting is intended to help me:

- Identify the age range and technological ability of the intended users
- Understand how the game may be used during school time
- Determine which features might be essential for the game
- Determine which features might be requested for the game

To help me find out as much as possible about my client and stakeholders and have a clear list of their needs and expectations. I have a questionnaire that consists of the questions I plan to ask and why I plan to ask them, including how I think it can help me to develop the best possible version of my game possible.

Interview questions and justifications:

Q1. What types of games have you enjoyed the most and why?

This question helps me understand what students already enjoy and will allow me to design my game in a way that aligns with their interests and keeps their engagement.

2. How often would you expect the students to play the game and for how long in a typical session?

Knowing the number of users and expected playtime provides a look into the required scale and pacing of my game. This will influence design decisions such as game/level length and the overall game structure.

3. Are there any features you believe are essential for the game to be suitable for your students?

This question ensures that the main requirements of my primary stakeholders are known and met

4. Would you prefer the game to be singleplayer or should it include multiplayer features?

This question informs me of a major structural decision in the games design. A single player game greatly reduces complexity, while multiplayer functionality would greatly alter the technical approach I would take to a fundamental level.

5. Is it important that the game can be paused, saved or exited at any point?

Since my game may be played during school time it is important to confirm whether students need the ability to stop playing immediately. This question helps determine the requirements for game saving progress and session timing.

6. Would optional guidance or hints be useful for students while playing the game?

This question looks at whether additional support systems should be added e.g. hints or guidance that could help the player understand the rules and mechanics of the game and how to help players with different ability levels.

7. Do you and your class currently use or play any other games in school or at home?

This question will help identify certain games that they might be familiar with. It will also provide some context for how my game may compare to other games students already know.

8. Are there any features from those games that you feel work particularly well for you?

By identifying successful features from existing games, I can adapt similar ideas into my game ensuring that the game feels intuitive and is enjoyable for the students.

9. Are there any elements you believe should be avoided when designing this game?

This question highlights potential risks with a games mechanics such as overly complex controls and interfaces. Anticipating these issues helps me make a safer design process.

10. Is there anything else you feel is important that I have not already asked about?

This final open-ended question ensures that there is no glaring information that I missed out and gives Mr Moores and the students an opportunity to raise any additional concerns and considerations that may influence the design of my game.

Interview transcript:

My client and I met up on Wednesday 4:00pm in the club room to talk about his requirements and expectations for my project

To make sure that I didn't miss any important details I used a transcriber to record my interview and write down what was said, here is a summarised transcript of the interview:

Andrew: hi guys, it's good to see you. I hope you're doing well Mr Moores, thanks for taking the time to speak with me. Would it be alright if I recorded our convocation?

Mr Moores: no problem.

Andrew: thank you

Q1. Andrew: what types of games have you enjoyed the most and why?

Students: I like games that have

Q2. Andrew: How often would you expect the students to play the game and for how long in a typical session?

Mr Moores: this would mainly be during lesson time, so not every day. A typical session would probably last around 10 to 20 minutes depending on the topic we are learning about.

Q3. Andrew: Are there any features you believe are essential for the game to be suitable for your students?

Mr Moores: the game needs to be easy to understand with clear instructions and simple menus so the students can focus on the programming aspects of the game rather than trying to learn the controls.

Q4. Andrew: Would you prefer the game to be singleplayer or should it include multiplayer features?

Mr Moores: I would strongly prefer a singleplayer experience as multiplayer games will steal their attention and a singleplayer format allows students to focus on the code without problems.

Q5. Andrew: Is it important that the game can be paused, saved or exited at any point?

Mr Moores: yes, that is very important in a school environment as the students will need to be able to stop playing immediately when the lesson ends or when we want to start the programming tasks.

Q6. Andrew: Would optional guidance or hints be useful for students while playing the game?

Mr Moores: I don't think optional hints would be very useful in this game, although I think that adding in instructions could help any students that are unfamiliar with this style of game

Students: adding them in wouldn't hurt and they can only really help us

Q7. Andrew: Do you and your class currently use or play any other games in school or at home?

Mr Moores: I'm sure many students play video games at home. However, a lot of these contain part that are not friendly to learn code, the main example would be that the code is closed source, so we aren't even able to view it which is obviously a problem for our lessons. Some sort of platformer game like Mario or Smashbros

Q8. Andrew: Are there any features from those games that you feel worked particularly well for you?

Mr Moores: clear objectives with visual feedback when something happens tends to work well, like when the player or an enemy takes damage, they flash red and a sound effect plays.

Q9. Andrew: Are there any elements you believe should be avoided when designing this game?

Mr Moores: overly complex mechanics should be avoided. Nothing that would pull too much attention away from the code of the game, if the game is too fun to students, they will find it hard to stay focused.

Q10. Andrew: Is there anything else you feel is important that I haven't already asked about?

Mr Moores: the games should be able to run on the school computer smoothly and without issue, since the students will be taking the game home it needs to be able to work with Windows and MacOS devices. None of the students use Linux so it doesn't need to be compatible with any distro.

One thing you could do is make the images for the player and enemies stand out from both each other and the background.

Andrew: don't worry Mr Moores I will make sure to do that.

Mr Moores: thank you Andrew.

Andrew: thank you Mr Moores, thanks for meeting with me guys and have a great rest of your week.

Review of the interview:

This interview formed a major component of my research as it allowed me to gather direct stakeholder requirements from my client, Mr Moores and his students. By analysing the transcript of the interview, I was able to extract both fundamental and non-fundamental requirements for the game which will directly impact the design of my game.

From the discussion there were several key design priorities that stuck out. Firstly, the game must be singleplayer as Mr Moores expressed concern that multiplayer functionality could lead to loss of focus within the classroom. This means that my game must have a PVE system rather than PVP mechanics.

Secondly, the game must support short play sessions (10-20 mins) which can be paused throughout. This will help shape the level designs and pacing of the game. Another important feature identified was an ability to change the difficulty of the game as different students have different levels of ability. Therefore, my game will have an option to change the difficulty to help any students that are struggling. This will keep the game challenging for more confident players while ensuring accessibility for other less confident students. Finally, the ability to pause the game at any time will be implemented to ensure the game does not interfere with any learning and studying they might be doing.

Overall, the interview allowed me to get a grasp of the general ideas for the game and clearly define the system and game requirements.

Questionnaire

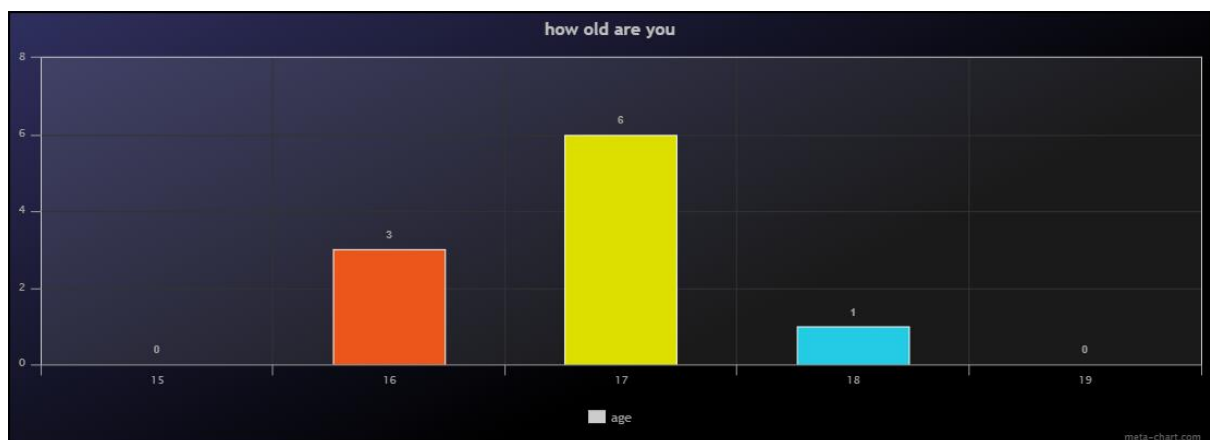
In addition to interviewing Mr Moores and his students I sent a questionnaire to them. A questionnaire was selected as the most efficient research method because:

- It allows responses to be collected remotely
- It ensures consistency in the questions asked
- It enables quantitative analysis through graphs/charts
- It reduces time constraints compared to conducting many interviews

Using questions like multiple choice and scaled responses I was able to gather measurable data regarding:

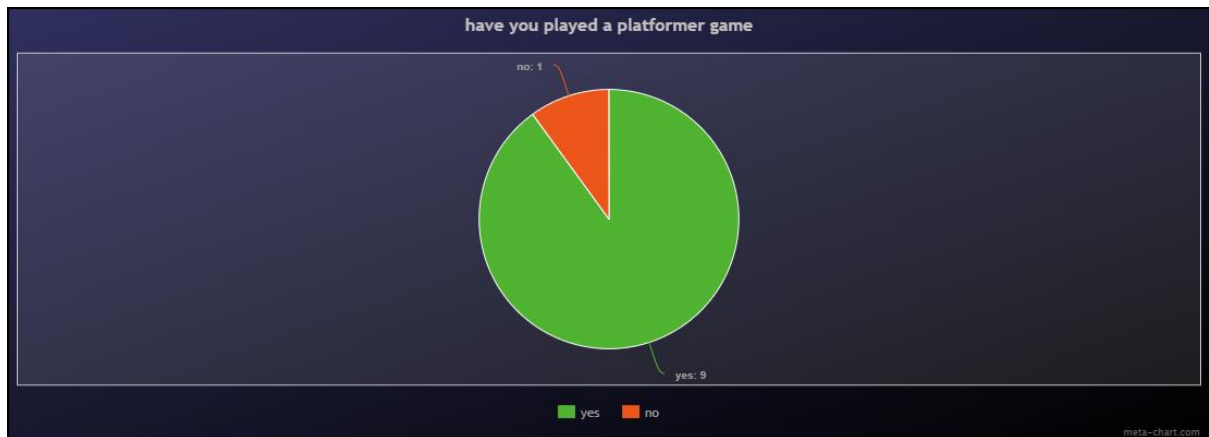
- Preferred game genres
- Attitudes towards hint systems
- Preferences regarding difficulty levels

This quantitative data also complements the qualitative insights gained from the interview well.



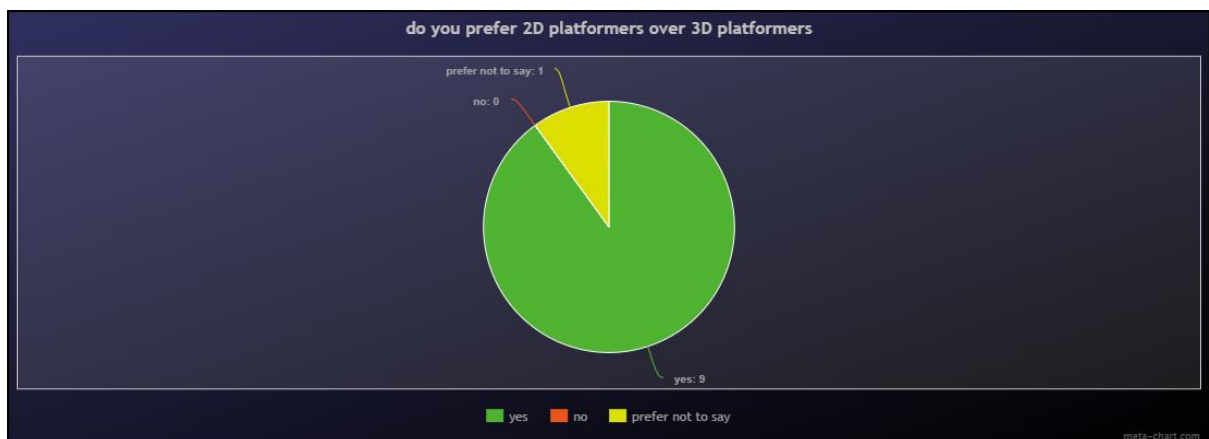
How old are you?

I asked this to understand the age range of the students. The results showed that most of them were 17, with some being 16 and 18. This shows that my game should mainly be designed for older teenagers in college.



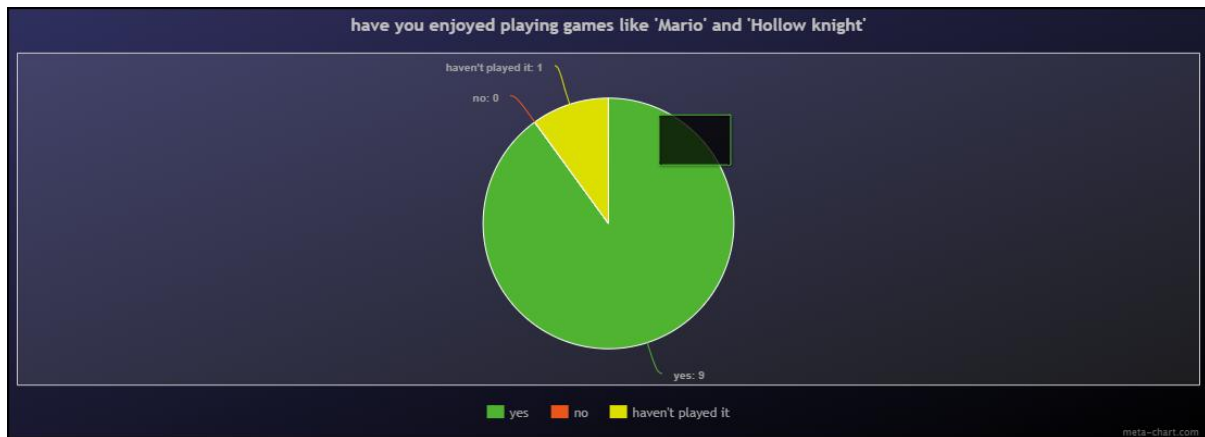
Have you played a platformer game?

I have asked this to see whether the students had experience with the type of game I am designing. The results showed that 9 people had played a platformer game before and 1 had not. This means most students understood the genre and could give useful feedback.



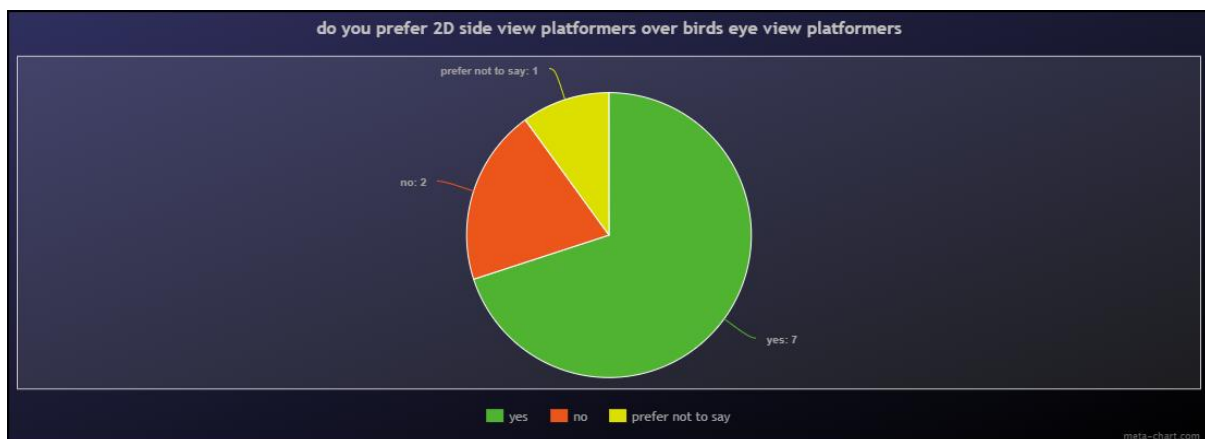
Do you prefer 2D platformers over 3D platformers?

I asked this to confirm whether a 2D or 3D game will be the right choice. The results showed that 9 people preferred 2D platformers, 1 preferred not to say and nobody said no. This means that I will be making the game a 2D platformer.



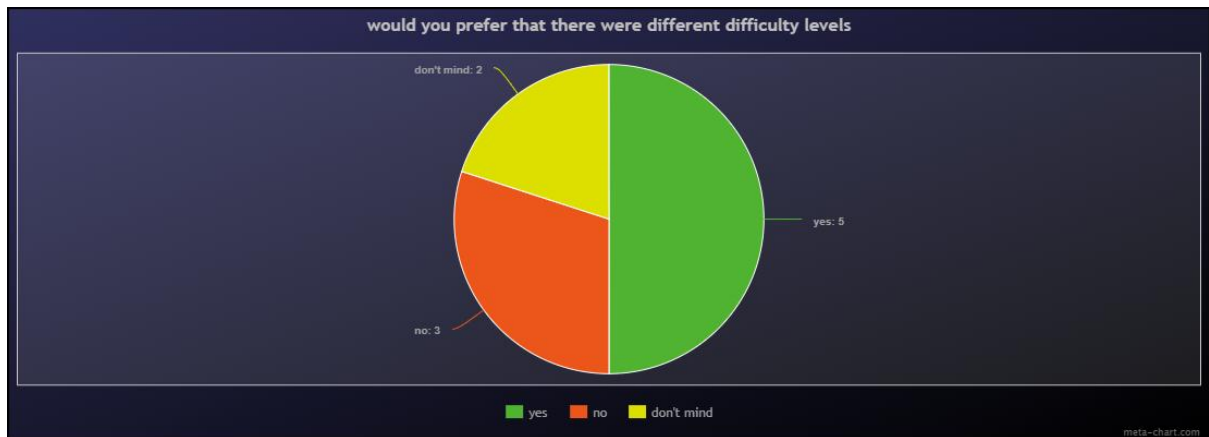
Have you enjoyed playing games like Mario and Hollow Knight?

I asked this because these games are similar to the type of platformer that I want to make. The results showed that 9 people said yes, 1 person had not played them and nobody said no. This shows that a platform game would appeal greatly to the students.



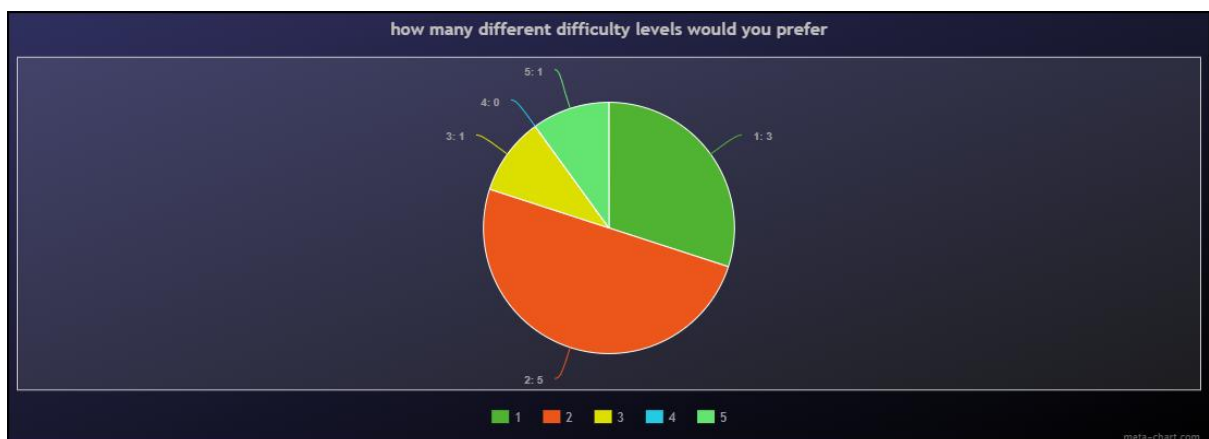
Do you prefer side view platformers over birds eye view platformers?

I asked this to decide the way that the game is displayed. The results showed that 7 people preferred 2D side view platformers, 2 said birds eye view and 1 preferred not to say. This shows that a side view platformer would be the best format for my game.



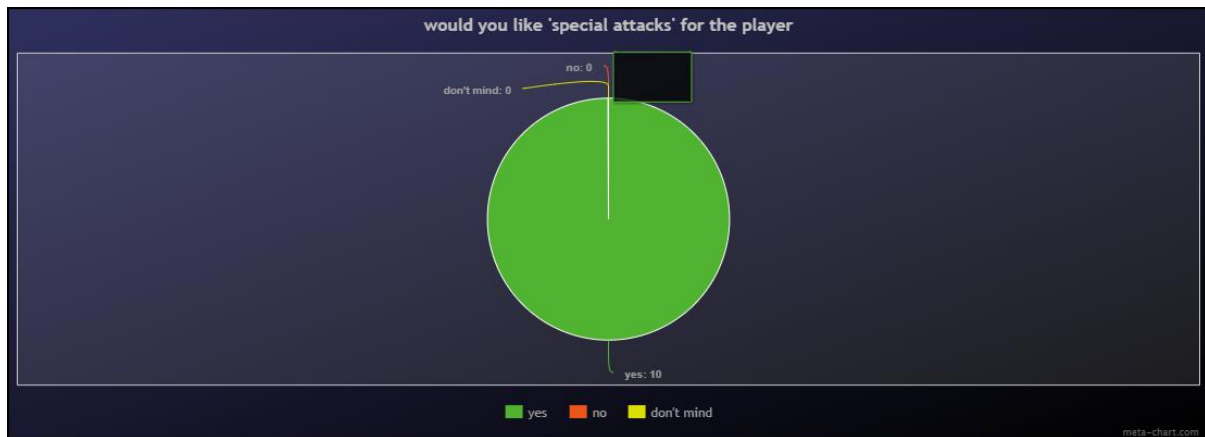
Would you prefer that there were different difficulty levels?

I asked this to check whether players actually wanted the different levels to have varying difficulty. The results showed that 5 people said yes, 3 said no and 2 did not mind. This shows that adding different difficulty would be useful but kept to the bare minimum.



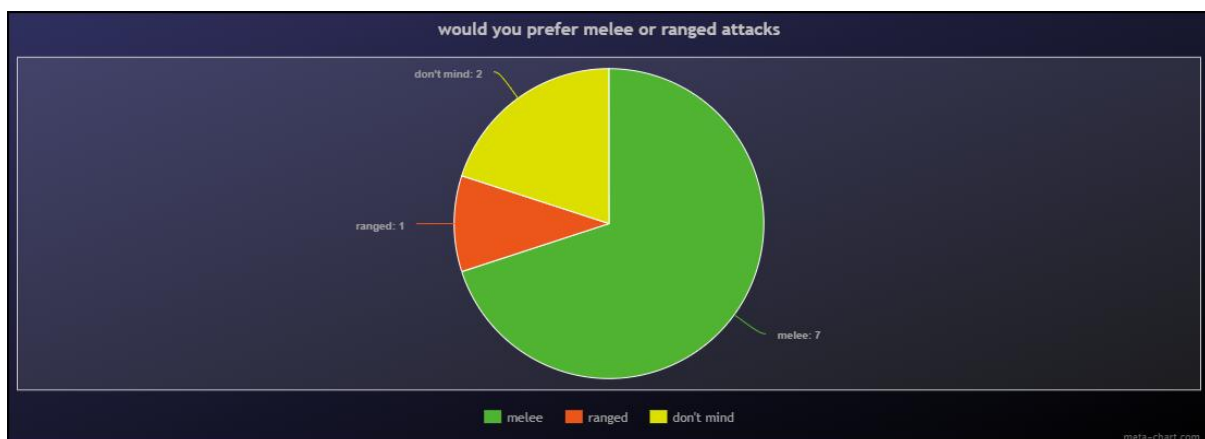
How many different difficulty levels would you prefer?

I asked this to decide how many difficulty options the game should have. The most popular answer was 2 difficulty levels (easy and hard), chosen by 5 people. Three people chose 1 difficulty level(normal), one person chose 3 (easy, normal and hard) and one person chose 5 (easy, less easy, normal, hard, extra hard). This shows that having 2 difficulty levels would suit the most students and stop my game from getting too complicated.



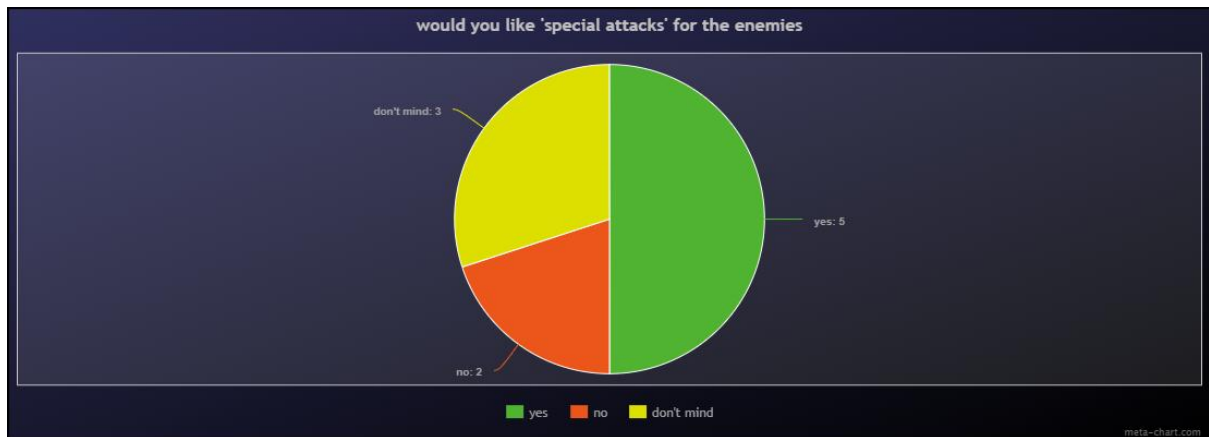
Would you like special attacks for the player?

I asked this to find out whether the character should have some more interesting combat options where all 10 students said yes. This shows that there is strong support for adding special attacks for the character, therefore it should be included in the game.



Would you prefer melee or ranged attacks?

I asked this to help decide the main combat style of the character. The results showed that 7 people preferred melee attacks, 1 preferred ranged attacks and 2 did not mind. This shows that melee attacks should be the main attack method, but as all students wanted a special attack, a ranged attack could be used as that.



Would you like special attacks for the enemies?

I asked this to decide whether enemies should have more advanced attack mechanics. The results showed that 5 people said yes, 3 did not mind and 2 said no. This shows that enemy special attacks would be well received by the majority but that I should not make them too powerful.

Overall conclusion

The questionnaire helped me understand what the students want from my game. Most respondents were 17 and had experience with platformers. The results show that they prefer a 2D side view platformer game with player special attacks, melee combat and simple difficulty options. Based on this feedback I should design my game to meet these requirements as closely as possible.

Evaluation of research methods

Another research group I considered conducting was a focus group, however I found that this method was not practical as a focus group would have required multiple participants working with each other simultaneously which difficult to set up and both time consuming and requires lots of resources.

Secondary research was also limited because, while there was general information available on 2D platformer games there is little data on how these games could be implemented to be academically useful as most news outlets and scientific reports work on the harmfulness of videogames rather than the benefit.

Therefore, primary research was the bulk of my study as it was the most appropriate method for gathering relevant and content specific knowledge.

Conclusion

The combination of interview and questionnaire data has provided both qualitative and quantitative data to help guide development decisions for my game. These findings reinforce the features that my game should have:

Success requirements and user requirements:

Feature	Explanation	Necessity	Justification
Movement	Basic up, down, left, right and jumping movement for the character and enemies	Necessary	In order for the students to be able to play the game the character and enemies need to be able to move
Wall jumping	when a player jumps from the ground to a wall, they will be able to jump again, off that wall	Necessary	My game will have platforms, so the character needs to be able to move through and around them
Wall grabbing	When a character moves next to a wall, they can hold on to it and hang from it	Necessary	The platforms will be large and difficult to get around so it is important that the players have a way to navigate through and around them
Player (basic) melee attack	A simple melee which will affect a small area in front of the player	Necessary	The character needs a way to defeat the enemies in order to progress into the next level
Player (special) attack	A more powerful attack with a cooldown, ranged and higher damage.	Necessary	This was requested by everyone and adds depth to combat and gives players additional options to learn from when they want to create an attack for themselves

Enemy (basic) melee attack	Standard enemies can damage the character when they make contact	Necessary	In order for there to be a challenge to the game, the enemies need to be able to eliminate the character and prevent progression between the levels from being too easy
Enemy (special) attack	some enemies may have unique attacks such as projectiles or area attacks	Desired	Special attacks increase variety between enemy types and make later levels more engaging
Difficulty options	Players can select different difficulty settings which affect player and enemy damage and health	Desired	Difficulty settings allow the game to appeal to a wider range of players with different skill levels whilst also improving accessibility for less skilled players
Save and quit	Players save progress automatically by moving on to the next level, to quit they pause gameplay and exit the game at any time	Desired	The game will need to be closed quickly in order to complete the main tasks in the lessons, however it is less important that the students can load back their progress as the main aim of the game is not progression, but lessons that they can learn from the code
Guidance system	Hints that explain where to go and what to do	Not important	Although it could be helpful for really inexperienced players, the controls and objectives are explained in the instructions menu and the game is simple enough to

			not need additional guidance
Special movement tech	Advanced movement mechanics such as dashing, sliding or air movement techniques	Not important	These features would make gameplay too complex and will pull attention away from learning
Different levels	Multiple levels with progressively difficult enemies	Desired	As my game has different difficulty options, there is not as much need to have lots of levels. However, it would add depths and make it more engaging so I will add it if I deem it necessary later on
Main menu	When the main menu is opened, 'settings', 'instructions' and 'quit' will be displayed on the screen	Necessary	The game requires the menu buttons, settings screen, instructions screen and window controls to function correctly. The main menu acts as the hub for overall control of the game.

Success criteria

Criteria	Requirement	Necessity
Character moves right when the 'D' or right arrow key is pressed	Ensures movement requirements are met	Necessary
Character moves left when the 'A' or left arrow key is pressed		
Character jumps when 'Space' key is pressed		

Gravity pulls the character down when they are not standing on a platform		
Character can't fall through platforms		
Enemies move randomly within designated areas		
Enemies can't move off their platform		
Enemies can't fall through platforms		
When the player is touching a wall the game registers it	Ensures wall jumping requirements are met	Necessary
When the player then presses 'space', the character jumps again but at an angle		
When the player moves into a wall while airborne, they can hold onto it	Ensures wall grabbing requirements are met	Necessary
While wall grabbing, the player does not fall		
Pressing the 'shift' key triggers a melee attack animation	Ensures player basic attack requirements are met	Necessary
Enemies within the attack range lose health		
Pressing the 'enter' key activates the special attack	Ensures player special attack requirements are met	Necessary

The special attack has a cooldown period after use		
Special attacks deal more damage than basic attacks		
Enemies damage the player when they make contact	Ensures enemies basic attack requirements are met	Necessary
The player loses health when hit by an enemy		
Special enemies can use unique attacks	Ensures enemies special attack requirements are met	Desired
Enemy special attacks damage the player correctly		
The player can select a difficulty setting before starting the game	Ensures difficulty options requirements are met	Desired
Enemy and player statistics change according to the selected difficulty		
The game can be paused during gameplay by entering the menu	Ensures save and quit requirements are met	Desired
The player can exit the game from the menu		
The player's level is being saved successfully		

Completing a level loads the next level successfully	Ensures different levels requirements are met	Desired
Each level contains different enemy placements and layouts		
The main menu is displayed when the game launches	Ensures main menu requirements are met	Necessary
Pressing the new game button starts the game		
Pressing the load game button loads the saved game		
Pressing the 'esc' key opens the settings menu and allows for features to be changed		
Pressing the instructions button opens the instructions screen		
Pressing the quit button closes the program		

Limitations

Although I believe that I can meet my client's requests and create a user friendly and enjoyable game that is suitable for school children, I have noticed there are several issues that I may come across. I have identified these limitations through the computational methods I have talked about above, as well as through my research into similar platformer games, the needs of my client and other stakeholders via research methods such as surveys.

One potential limitation of my game could be that if the program is overloaded with requests e.g. repeatedly triggering jump actions or interactions with enemies or platform

elements in quick succession it could result in the game behaving unexpectedly or breaking entirely.

Some examples of other limitations are:

Limitation	Explanations
Guidance system	I won't have an in-game guidance system as I will have instruction on the main menu that will run through the inputs and what you must do to complete a level
language	My game will be in English and so will be inaccessible for non-English speakers as I will not put in options to change the language because it is not a high priority to make it available in multiple languages as my stakeholders are all fluent in English
Special movement	I could have more complex movement mechanics like the crystal dash from the game 'Hollow Knight', but I will limit the movement to left, right and a double jump where the player can jump midair. This is so I can priorities refining the main movement tech and so the students can focus better.
Number of levels	I will only create a few levels for my game as I don't want to use the majority of my development time creating different levels. However, I do want to have more than one level so that I can have a more diverse difficulty progression on top of the difficulty options to show the students.
Number of difficulty options	I will only give 2 difficulty options to not take away from the actual aim of the game, to teach the students about coding. There only needs to be the 2 settings to teach them a method of how to implement the difficulty options.
Save and load functionality	The students won't be able to save multiple runs on the computer as that could make the game run significantly worse on older devices with less storage. For this reason, I decided that make only one save file per game.
Software compatibility	My project will be coded using greenfoot and android based devices will not be able to install and run greenfoot. This includes Chromebooks. Since this was not a

	requirement for the program it is a non-issue
--	---

Hardware and software requirements

Hardware requirements

The recommended hardware requirements of a device used to run the game would be:

- **Central Processor Unit (CPU):**
- **Memory:** this is the rough size of the program I expect to create and the images I plan to use in my game
- **Graphical Processing Unit (GPU):** integrated graphics
- **Audio controller:** Realtek
- **Speakers and headphones:** any, in order to hear the sound effects
- **Keyboard:** a keyboard is required for the functionality of the game
- **Mouse:** a mouse is required for the functionality of the game

Software requirements

My project will be coded using greenfoot. The Windows version of Greenfoot will run on 64-bit installations of Windows 7 or later, on machines with an x86 architecture. If you want to use Greenfoot on a 32-bit installation, then version 3.5.4 is the last version that would run on a 32-bit system.

The Macintosh version of Greenfoot will run on any modern Mac. The regular Mac version requires a 64-bit Intel compatible processor. The legacy package of Greenfoot 2.4.2 should run on any Intel compatible processor but also requires Java 6 to be installed separately. The security changes in macOS Catalina may make it hard to run older Greenfoot versions, version 3.6.0 should be compatible with the changes that were made.

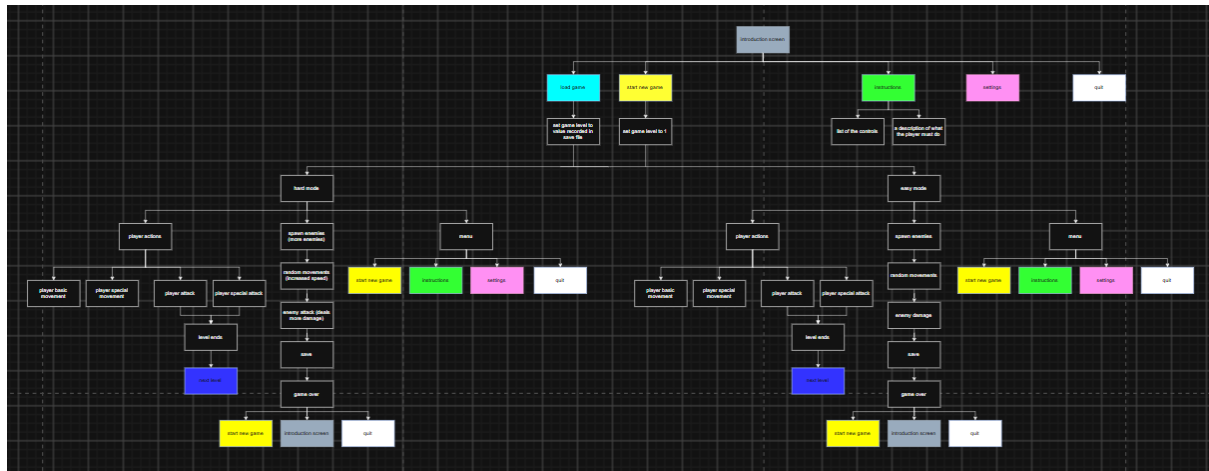
Should I put this: <https://www.greenfoot.org/doc/faq>

Design stage

Decomposition of the program

I have used my success criteria and the computational method of decomposition to divide my project up into smaller and more manageable sub problems. This will help me when developing my program as I am able to tackle smaller parts better and more efficiently.

I have displayed the subsections of my project in the structure diagram below:



Breakdown of each module

Introduction screen:

This will be the first screen that the player sees when launching the game. It will contain buttons for load game, start new game, settings, instructions and quit. The layout provides a simple way for players to navigate the games functions. I have chosen this design because it allows players to quickly access the different parts of the game.

Main game screen:

Once the player presses the start new game button the options for hard and easy mode will be shown, the player picks between the two then the main game screen is displayed. This is where the player controls the main character and explores the level. The screen contains the player, platforms, enemies and the user interface (player's health, enemies left etc).

Player movement:

The player can move left and right using the A and D keys as well as the arrow keys, jump to reach higher platforms using the 'space' button. The game also has wall grabbing and wall jumping which lets the me have more freedom with platform placement and increases the diversity of the different levels.

These mechanics encourage different strategies while demonstrating more advanced programming concepts such as collision detection that is different depending on the side that the player is on, physics and state management.

Player combat system:

The player can attack enemies using a basic melee attack which damages any enemies that are directly in front of the character to defeat enemies and progress to higher levels. As the game develops. Combat introduces concepts like hitboxes, collision detection, cooldown timers and animation matching.

Enemy system:

Enemies move about specific areas of each level with random movements. Different enemy types can have unique attacks which increases the difficulty of the game. This demonstrates the use of object-oriented programming by allowing multiple enemy classes to inherit common behaviours while implementing their own unique functionality.

Easy mode:

Easy mode is designed for players who are new to platform games or want a more relaxed experience. Fewer enemies are spawned throughout each level and player hp is higher which makes the game more forgiving. I have included this mode so that players of all skill levels can enjoy the game while also teaching them how to implement different difficulty levels.

Hard mode:

Hard mode is intended for experienced players looking for a greater challenge. More enemies appear in each level, they move faster and their attacks deal more damage, requiring more precise movement and combat. I have included this mode to increase replayability while also teaching them how to implement different difficulty levels.

Level progression:

When the player reaches the end of a level the next level is loaded automatically and the level they have reached is saved. Each level has different platform layouts and enemies gradually increasing the difficulty. Separating the game into individual levels also makes the code modular and easier to maintain and expand if needed.

Settings screen:

The Settings screen allows players to customise parts of the game such as the volume of the music and sound effects. Having adjustable settings improves accessibility and displays how configuration values can be stored and used throughout a program.

Instructions screen:

The Instructions screen explains the controls, combat system and objective of the game. This allows new players to quickly understand how to play without requiring outside assistance making the game more accessible to the students.

Menu:

At any point while playing the game the player can pause the game by opening the menu. From this menu they can resume playing, open the instructions / settings or quit the game. This is particularly useful in a classroom as it allows the students to quickly stop playing if required.

Quit:

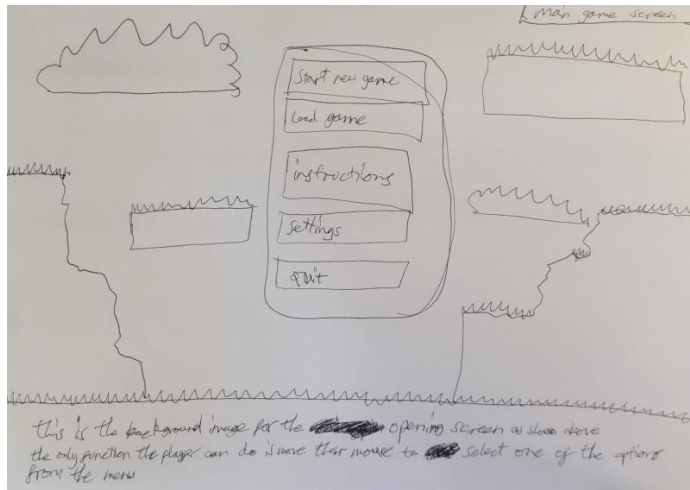
There are two ways to close the game window, it can either be done using the close button in the top corner of the screen or using this button in the menu. Although the close button in the top corner is available throughout the game, I have added the button onto the deploy screen as well to make the option clear as it is important for my client that the users are able to close the game quickly and easily.

End screen:

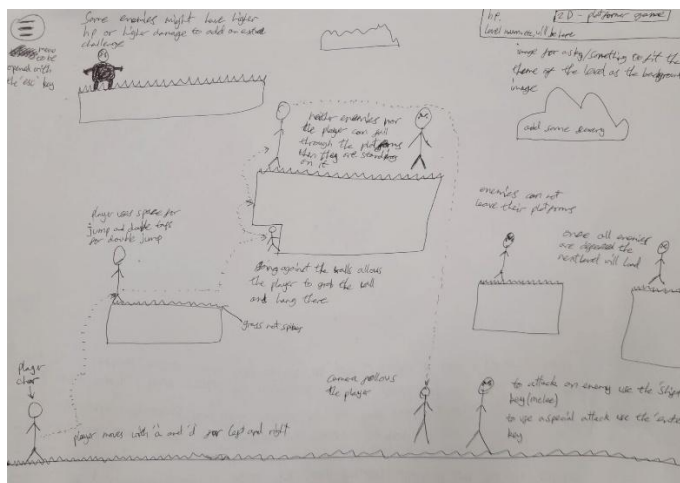
When the player loses all of their lives or completes the final level, the end screen is displayed. This screen has a congratulatory/defeated message, gives options to play the game again, return to the introduction screen or quit. Including these options improves the flow of the game while also demonstrating how different game states can be managed within a program.

GUI

An initial mock-up of my game's opening screen:



An initial mock-up of my games play screen:



Font style and size

I will use a simple font that is easy to read like 'Calibri' or 'times new roman' rather than a decorative font that could make text harder to read. Important text such as menu button labels will use a larger size to make it easier to see and read and I will keep the same font style throughout my game so the screens look consistent.

Layout and positioning

The main menu buttons are placed in the middle of the screen so they are quickly seen. The buttons are also placed in a vertical list making it easy to read and select and also because these placements are the standard for games.

During gameplay the centre of the screen is kept clear for the player, enemies and platforms. Information such as health is placed near the top right corner of the screen so it can be seen without covering the game.

Buttons or interfaces that perform similar actions are kept in similar positions on different screens. This makes the interface feel intuitive and easier to learn.

Special enemies' placement:

Enemies that have high health or damage that cause a significant challenge for the player will be placed on platforms that are further from the spawn point of the character compared to the standard enemies to prevent the player from being instantly killed and to show the students how coding this is possible

Opening screen

The opening screen will display the name of the game prominently using a large and clear font. The main menu will contain obvious buttons such as 'Start New Game', 'Load Game', 'Instructions', 'Settings' and 'Quit'.

The background will be designed to fit the game's overall visual style without making the text or buttons difficult to see. I will avoid using an overly bright background behind important information so that the user can see it clearly.



Image from: <https://stock.adobe.com/search?k=%22platform+game%22> (14/09/2026, 13:44)

Background

The background will use a simple 2D platformer style with floating platforms and clear scenery which will not be too detailed because the player and enemies need to stand out. The colours will have enough contrast to make the platforms, player and enemies easy to see. I will use the same visual style across the menu and gameplay screens so my game looks consistent. The background is decorative and will not contain objects that are interactive so I will avoid adding background images that look like platforms or enemies because this could confuse the player.

Usability features

Since the main users of my game will be students, usability is extremely important to my design. A specific example of this is clear and bold colours and visual indicators throughout the game. Different elements such as the player, enemies and platforms will look different so that players can quickly determine what they are looking at. This is particularly important during combat where clear visual feedback is essential to show the player which character is theirs or an enemy, when an attack has been performed or when they have taken damage etc.

I also want to have animations to provide a better experience when the player performs actions like jumping, attacking, taking damage or defeating an enemy. However, these animations will not have over the top flashing effects. This helps make the game more accessible to people with epilepsy while still providing better visual feedback to the player.

Colour scheme

The colour scheme of my game will be designed to make important objects easy to identify. By using a mixture of bold and mute colours for elements such as the player, enemies, platforms and interactive objects so that they can be distinguished from the background and each other.

I will also consider the readability of text when selecting colours. Important information such as health, instructions and menu options will have enough contrast against the background to ensure that it can be read easily.

I searched through a colour palette generator to find some colours that I thought would work well in my game:





Colour palettes from: <https://coolers.co/palettes/trending> (14/09/2026, 13:24)

Player and enemy sprites / animations

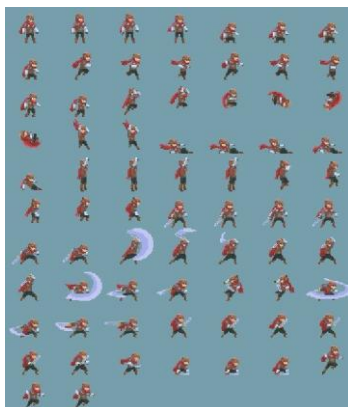
Player and enemy sprites

The player and enemies will use visually different sprites so that they can easily be identified during gameplay. This is important because the player needs to be able to easily pick out their character from the enemies while moving through a level.

Animations

Animations will be used to make the player's actions clear. e.g. the player will have different animations for standing, running, jumping and attacking. Enemies will also have animations that show their movement and attacks. Visual feedback will be provided when important events occur e.g. the player and enemies will briefly flash red and play an animation when they are hit or defeated. These effects will help players easily understand what has happened without requiring additional text or instructions.

The game will also use visual indicators for important information such as the player's health and current level. This means that the player can understand their current position without having to pause the game.



Player sprite/animation: <https://pinterest.com/ideas/animated-pixel-characters/957249094494> (14/09/2026, 13:50)



enemy sprite/animation: <https://www.shutterstock.com/search/enemies-character-animation> (14/09/2026, 13:58)

Buttons

Each button will be named according to its function e.g. start a new game Settings, Instructions, Save and Quit. This should make the interface easy to understand. The buttons will also provide visual feedback when the player moves their cursor over them, the button may change size or colour when hovered over which will show the player that the button is being interacted with.



Image from:

<https://imgs.search.brave.com/gsiQLpTRgV4qvr1yxhnh0FyQEHgKWX6iVKGxoXO3i4w/rs:fit:500:0:1:0/g:ce/aHR0cHM6Ly9zdGF0/aWMudmVidGVlenku/Y29tL3N5c3RlbS9y/ZXNvdXJjZXMvdGh1/bWJueWlscy8wMzUv/MzQ3LzEyMC9zbWFs/bC9zdG9uZS1idXR0/b25zLWFuZC1ib2Fy/ZHMTm9yLWdhbWUt/dWktZ3VpLWVsZW1l/bnRzLWlzb2xhdGVk/LW9uLXdoaXRiLWJh/Y2tncm91bmQtY2Fy/dG9vbi1zZXQtb2Yt/Z3JheS1zdG9uZS1i/YW5uZXJzLW1lbnUt/YnV0dG9ucy1hbmQt/YXJyb3dzLWZvci1t/b2JpbGUTZ2FtZS12/ZWN0b3luanBn> (14/09/2026, 14:07)

Key variables, data structures and classes

Key variables

Name	Data type	scope	Purpose and justification
screenWidth	Integer	Public	Stores the width of the game window in pixels
screenHeight	Integer	Public	Stores the height of the game window in pixels
playerX	Integer	Public	Stores the player's horizontal position on the screen
playerY	Integer	Public	Stores the player's vertical position on the screen
playerHealth	Integer	Public	Stores the player's current health. This is reduced when the player is damaged by an enemy

playerSpeed	Integer	Public	Stores how quickly the player can move across the level
playerVelocity	Integer	Public	Stores the player's current movement speed, including movement caused by gravity and jumping
Gravity	Integer	Public	Stores the amount of downward force applied to the player when they are in the air
jumpPower	Integer	Public	Stores the amount of force applied when the player jumps
gameState	String	Public	Stores the current state of the game, such as the main menu, gameplay, settings or game over screen
difficulty	String	Public	Stores the difficulty selected by the player (easy or hard)
levelNumber	Integer	Public	Stores which level the player is currently playing
enemyDamage	Integer	Public	Stores the amount of damage an enemy can cause to the player. This can change depending on the difficulty
enemyCount	Integer	Public	Stores the number of enemies that are currently active in the level
attackCooldown	Integer	Public	Stores the remaining cooldown time before the player's special attack can be used again

gamePaused	Boolean	Public	Stores whether the game is currently paused
playerOnGround	Boolean	Public	Stores whether the player is currently standing on a platform. This is used to control jumping
wallContact	Boolean	Public	Stores whether the player is touching a wall. This is used for wall jumping and wall grabbing

Key images and sounds

Name	Purpose and justification
mainMenuImage	Stores the background image used for the main menu
backgroundImage	Stores the background image used during gameplay
playerSprite	Stores the images used to display the player character
enemySprites	Stores the images used for the different enemy types
platformImage	Stores the images used to display platforms within the levels
buttonImage	Stores the images used for buttons throughout the game
jumpImage	Stores the images used for the player's jumping animation
attackImages	Stores the images used to create the player's attack animation
enemyAttackImages	Stores the images used for enemy attack animations
damageImage	Stores the visual effect shown when the player takes damage
deathImage	Stores the animation or image shown when an enemy is defeated or the player loses

endScreenImage	Stores the background image used for the game over or end screen
jumpSound	Stores the sound played when the player jumps
attackSound	Stores the sound played when the player performs an attack
hitSound	Stores the sound played when an attack successfully hits an enemy
damageSound	Stores the sound played when the player takes damage
music	Stores the background music played during the game

These variables and data structures will allow the different parts of my game to communicate with each other. They will also make the program easier to organise because information such as the player's position, health, enemies and current level can be stored and updated separately.

Classes

Class diagram

The diagram below shows the attributes and methods of the classes I plan to create for my game. My game is made in Greenfoot using Java and object oriented programming. Different parts of the game are in separate classes that make sense so that each class has a clear purpose making modular and so easier for the students to understand what does what. e.g. the player class controls the player while enemy class controls enemy behaviour.

Some classes will inherit from other classes when they share similar features e.g. different enemy classes can inherit from a main enemy class which means that common code such as health, movement and taking damage does not need to be repeated.

.....add class diagrams

Approach to Testing

I have numbered and written short descriptions of the tests I plan to complete. I have done this to ensure that while creating my game I focus on the requirements of my client and to

make sure that I maintain the focus of my project using my success criteria. I think that this is useful because it will help me manage my time and reduce the number of errors that can build up.

I have created two test plans. The first is the iterative testing that I plan to complete while developing my game. The second plan includes the tests I plan to complete on my final game to check that I have successfully completed all of my success criteria.

Iterative test plan

Test number	Type of test	Test data	purpose	Expected outcome
1	normal	Run program	To check that my game opens correctly and the game window appears	My game opens without errors. The window has the correct size and background
2	normal	Keyboard input	To check that the main menu loads correctly	The main menu appears with all buttons and text in the correct positions
3a	normal	Mouse click	To check that the play buttons work	Pressing 'start new game' starts my game and loads the first level
b	normal	Mouse click	To check that the play buttons work	Pressing 'load game' loads the level in the file
4	normal	Mouse click	To check that the instructions button works	Pressing the instructions button displays the instructions screen
5	normal	Mouse click	To check that the player can return from the instructions screen	Pressing the back button returns the player to the main menu

6	normal	Mouse click	To check that the settings button works	Pressing the settings button displays the settings screen
7	normal	Mouse click	To check that the player can return from the settings screen	Pressing the back button returns the player to the main menu
8	normal	Mouse click	To check that the settings button works	Pressing quit closes the game window and stops the program
9	normal	Keyboard input	To check that the player character moves left	When the left movement key is held the player moves left
10	normal	Keyboard input	To check that the player character moves right	When the right movement key is held the player moves right
11a	normal	Jump key	To check that the player can jump	When the jump key is pressed the player moves upwards and then falls back towards the ground
11b	boundary	Jump key	To check that the player can double jump	Pressing the jump key while the player is in the air does allows another jump
12	normal	Player movement	To check that gravity affects the player	When the player is not standing on a platform they fall downwards
13a	normal	Player movement	To check that the player can land on platforms	When the player falls onto a platform they stop on top of it instead of falling through it

13b	boundary	Player movement	To check collisions at the edge of a platform	The player can stand close to the edge of a platform and falls when they move beyond it
14	normal	Player movement	To check that the player cannot move through solid objects	The player stops when they collide with a solid wall or other solid level object
15	normal	Player movement	To check that the player's animation changes when moving	The correct walking or running animation is shown while the player moves
16	normal	Player movement	To check that the player faces the correct direction	The player faces left when moving left and faces right when moving right
17	normal	Level start	To check that platforms and other level objects load correctly	All platforms, walls and level objects appear in their expected positions
18	normal	Game running	To check that enemies appear in the correct places	Enemies load in their expected starting positions when the level begins
19	normal	Game running	To check that enemies move correctly	Enemies follow their intended movement pattern while my game is running
20a	normal	Player and enemy collision	To check what happens when the player touches an enemy	The player loses a life when they collide with an enemy
20b	boundary	Player and enemy collision	To check that one collision does not	A single collision only removes one life

			remove several lives immediately	
21	normal	Lose life	To check that the lives display updates correctly	When the player loses a life the number of lives shown on the screen decreases by one
22	normal	Reach level end	To check that the level can be completed	When the player reaches the end point the level is marked as complete and the next game state is loaded
23	normal	Complete level	To check that the next level loads correctly	The next level loads with the player, platforms and enemies in their correct starting positions
24	normal	Complete final level	To check that completing all levels triggers the correct ending	When the final level is completed the completion screen is displayed
25	normal	Lose all lives	To check that the game over state works correctly	When the player has no lives remaining the game over screen appears and normal player movement stops
26	normal	Level change	To check that player values are handled correctly when moving between levels	Lives are not reset upon entering a new level
27	normal	Player movement	To check that the camera follows the player if the level is	The camera moves through the level while keeping the player visible

			larger than the window	
28	boundary	Player movement	To check that the camera does not move outside the level	At the start and end of the level the camera stops at the level boundary
29	normal	Sound test	To check that the correct sound plays when the player jumps	The jump sound plays once when the player jumps
30	normal	Sound test	To check that the correct sound plays when the player is damaged	The damage sound plays when the player loses a life

Post development test plan

After I have finished coding my game I will carry out the final testing myself. This will focus on robustness and performance rather than testing the individual features again. I will check that the finished game works correctly in different conditions and does not crash when unexpected inputs are used.

I will also ask other people to beta test my game. They will be given the finished version without being shown how everything works. They will play the game normally and report any problems they find. I will ask them about controls, difficulty, performance, instructions and whether anything was confusing or did not work as expected.

Purpose of test	Test data	Expected outcome	Actual outcome
To check that my game works on different computers	Run my game on several Windows computers	My game should open and run correctly on each computer	
To check that my game works on	Run on a lower specification and	My game should remain playable and should not	

computers with different specifications	higher specification computer	crash or become extremely slow	
To check that repeated key presses do not cause errors	Rapidly press movement, jump and attack keys	My game should continue to work without crashing or behaving unexpectedly	
To check that several controls can be used at the same time	Hold movement while repeatedly jumping and attacking	The controls should respond correctly and my game should remain stable	
To check that repeatedly restarting my game does not cause problems	Restart my game many times	Each new game should start correctly without errors or old data remaining	
To check that repeatedly moving between screens does not cause errors	Move between the menu, instructions, game and end screen many times	Each screen should load correctly and my game should not crash	
To check performance during busy parts of the game	Have several enemies attacking and moving at once	My game should remain responsive with no major slowdown	
To check that unusual player actions do not break my game	Move against walls, jump repeatedly at edges and attack while moving	My game should handle these actions without errors	
To check that the game can be played from start to finish without restarting	Complete the whole game in one session	My game should run from the main menu to the end screen without crashing	
To check that the game still works after	Play several games one after another	Performance should remain consistent and	

being played repeatedly		previous games should not affect new ones	
To check that another user can understand how to play without help	Give the finished game to another person	The user should be able to understand the controls and play the game using the instructions provided	
To identify problems that I did not find during development	Several people play my game normally	Testers should report any errors, confusing parts or unexpected behaviour they find	

Development stage

Algorithmic solutions

Algorithms

This section has the necessary algorithms to produce my game. There are separate diagrams and explanations for each of my different algorithms that I plan to include in my program and a final flowchart to show the main game loop

loadLevel

Purpose:

This algorithm loads the required level data and creates the platforms, enemies and objects needed for the player to interact with.

Pseudocode:

```

FUNCTION loadLevel(levelNumber)
    open level file using levelNumber
    FOR each tile in level file
        IF tile is a platform THEN
            create platform object
            add platform to platform list

        ELSE IF tile is an enemy THEN
            create enemy object
            add enemy to enemy list

        ELSE IF tile is a collectible THEN
            create collectible object
            add collectible to collectible list
        END IF
    END FOR
    set player starting position
END FUNCTION

```

Flowchart:

createPlayer

Purpose:

This algorithm creates the player character and sets their starting statistics.

Pseudocode:

```

FUNCTION createPlayer()
    create player object
    set player x position
    set player y position
    set player health
    set player movement speed
    set player attack damage
    set player special attack cooldown
    return player object
END FUNCTION

```

Flowchart:

playerMovement

Purpose:

This algorithm allows the player to control their character using keyboard inputs.

Pseudocode:

```
FUNCTION playerMovement()  
    check keyboard input  
    IF left key pressed THEN  
        move player left  
  
    ELSE IF right key pressed THEN  
        move player right  
    END IF  
  
    IF jump key pressed AND player can jump THEN  
        apply jump force  
    END IF  
  
    apply gravity  
    update player position  
END FUNCTION
```

Flowchart:

wallGrab

Purpose:

This algorithm allows the player to temporarily attach to walls when falling beside them.

Pseudocode:

```
FUNCTION wallGrab()  
    check if player touches wall  
    IF player touching wall AND player is falling THEN  
        reduce falling speed  
        set wall grabbing to TRUE  
  
    ELSE  
        set wall grabbing to FALSE  
    END IF  
END FUNCTION
```

Flowchart:

wallJump

Purpose:

This algorithm allows the player to jump away from walls.

Pseudocode:

```
FUNCTION wallJump()  
    IF player is wall grabbing THEN  
        IF jump key pressed THEN  
            apply upward force  
            move player away from wall  
            disable wall grabbing  
        END IF  
    END IF  
END FUNCTION
```

Flowchart:

collisionDetection

Purpose:

This algorithm checks whether the player has collided with platforms or other objects and prevents them from moving through solid objects.

Pseudocode:

```

FUNCTION collisionDetection()
    check player's horizontal movement
    IF player collides with a wall THEN
        stop horizontal movement
    END IF

    check player's vertical movement
    IF player is falling onto a platform THEN
        place player on top of platform
        stop falling
        set player as grounded
    END IF

    IF player hits the bottom of a platform THEN
        stop upward movement
    END IF
END FUNCTION

```

Flowchart:

basicAttack

Purpose:

This algorithm controls the player's standard melee attack and determines whether an enemy is within the attack range.

Pseudocode:

```

FUNCTION basicAttack()
    IF attack key is pressed THEN
        create attack hitbox
        play attack animation

        FOR each enemy
            IF enemy collides with attack hitbox THEN
                reduce enemy health
            END IF
        END FOR
    END IF
END FUNCTION

```

Flowchart:

specialAttack

Purpose:

This algorithm controls the players special attack and prevents it from being used continuously by using a cooldown.

Pseudocode:

```
FUNCTION specialAttack()
  IF special attack key is pressed THEN
    IF cooldown = 0 THEN
      create special attack hitbox
      play special attack animation
      ► deal special attack damage
      set cooldown timer
    END IF
  END IF
  reduce cooldown timer
END FUNCTION
```

Flowchart:

spawnEnemies

Purpose:

This algorithm creates enemies at the locations specified by the level. The number of enemies can change depending on the selected difficulty.

Pseudocode:

```
FUNCTION spawnEnemies(difficulty)
  read enemy locations from level
  ► FOR each enemy location
    create enemy
    IF difficulty = EASY THEN
      use normal enemy damage

    ELSE IF difficulty = HARD THEN
      increase enemy damage
    END IF

    add enemy to enemy list
  END FOR
END FUNCTION
```

Flowchart:

enemyMovement

Purpose:

This algorithm controls how enemies.

Pseudocode:

```
FUNCTION enemyAI()
  FOR each enemy
    check distance between enemy and player
    IF player is within detection range THEN
      move towards player
      IF player is within attack range THEN
        stop moving
        prepare attack
      END IF
    ELSE
      continue patrol movement
    END IF
  END FOR
END FUNCTION
```

Flowchart:

enemyAttack

Purpose:

This algorithm allows enemies to damage the player when they successfully attack.

Pseudocode:

```
FUNCTION enemyAttack(enemy)
  IF player is within enemy attack range THEN
    create enemy attack hitbox

    IF player collides with attack hitbox THEN
      reduce player health
      play damage animation
    END IF
  END IF
END FUNCTION
```

Flowchart:

checkLevelComplete

Purpose:

This algorithm checks whether the player has reached the end of the current level.

Pseudocode:

```
FUNCTION checkLevelComplete()  
    IF player reaches level exit THEN  
        IF more levels exist THEN  
            load next level  
        ELSE  
            display completion screen  
        END IF  
    END IF  
END FUNCTION
```

Flowchart:

saveAndQuit

Purpose:

This algorithm saves the players current progress so the game can be closed and continued later.

Pseudocode:

```
PROCEDURE saveAndQuit()  
    store current level  
    store player position  
    store player health  
    store score  
    write information to save file  
    close game  
END PROCEDURE
```

Flowchart:

loadGame

Purpose:

This algorithm retrieves previously saved information and restores the player's progress.

Pseudocode:

```
FUNCTION loadGame()  
    check if save file exists  
    IF save file exists THEN  
        read save file  
        load saved level  
        load player information  
        load difficulty  
  
    ELSE  
        start new game  
    END IF  
END FUNCTION
```

Flowchart:

mainMenuScreen

Purpose:

This algorithm displays the main menu and allows the player to choose between starting the game, selecting difficulty, viewing instructions or quitting.

Pseudocode:

```
PROCEDURE mainMenuScreen()  
    load title  
    load play button  
    load difficulty button  
    load instructions button  
    load quit button  
    IF play button is pressed THEN  
        start selected game mode  
    END IF  
  
    IF difficulty button is pressed THEN  
        open difficulty menu  
    END IF  
  
    IF instructions button is pressed THEN  
        open instructions screen  
    END IF  
  
    IF quit button is pressed THEN  
        close game  
    END IF  
END PROCEDURE
```

Diagram:

endScreen

Purpose:

This algorithm controls what happens when the player completes the game or loses.

Pseudocode:

```
PROCEDURE endScreen()
    IF player has completed all levels THEN
        display "Game Complete"
    ELSE
        display "Game Over"
    END IF

    load main menu
    IF start new game is pressed THEN
        start new game
    END IF

    IF instructions is pressed THEN
        open instructions
    END IF

    IF settings is pressed THEN
        open settings
    END IF

    IF save and quit is pressed THEN
        saveAndQuit()
    END IF
END PROCEDURE
```

Diagram:

mainGameLoop

Purpose:

This is the main algorithm that controls the overall operation of my game. It repeatedly checks inputs, updates the game and displays the current state.

Pseudocode:

```
PROCEDURE mainGameLoop
  start game
  WHILE game is running
    check player input

    IF player is on main menu THEN
      mainMenuScreen()
    END IF

    IF player is playing THEN
      updateGameScreen()
    END IF

    IF player reaches the end of a level THEN
      loadLevel(levelNumber)
    END IF

    IF player loses all health THEN
      endScreen()
    END IF

    IF player completes all levels THEN
      endScreen()
    END IF

    IF player chooses save and quit THEN
      saveAndQuit()
    END IF
  END WHILE
END PROCEDURE
```

Flowchart:

1. Setting up the greenfoot window and background

In this section I write the code required to make the main game window. This will be the area where my platform game is played on

<pre>public static final int screenWidth = 1000; public static final int screenHeight = 600;</pre>	I created variables for the width and height of the game screen. I used variables so the size can easily be changed later without changing other parts of the code and using final which prevents it from being changed
<pre>public GameWorld() { super(screenWidth, screenHeight, 1); }</pre>	I have created the game window using the pre-defined variables above for the height and width
<pre>GreenfootImage background = new GreenfootImage("background.png"); background.scale(screenWidth, screenHeight); setBackground(background);</pre>	These lines, create a background using the image 'background.png', set the scale of the image to the game window size, and set the background to that object. (for now it is just a white box)

Test 1:

Purpose: to check that my game opens correctly and the game window/background appears

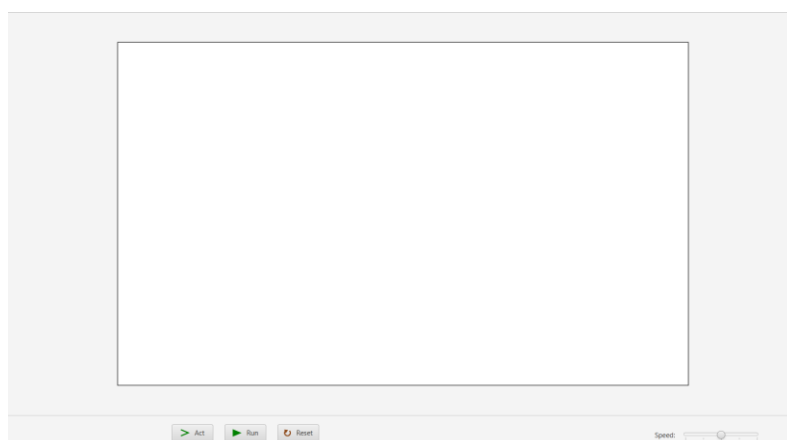
Type of test: normal

Test data: run program

Expected outcome: my game opens without errors. The window has the correct size and background

Actual outcome: success

The screenshot below is proof that my game window opened and that there were no errors when I ran the program. When closing the game window it also went well because there were no errors



2. Creating the basic game layout (load platforms)

<pre>private void loadLevel() { if (GameSettings.levelNumber == 1) { loadLevelOne(); } else { loadLevelTwo(); } }</pre>	Since this is the beginning of my development I have only added in a simple level system to help with the initial structure of my code. This part is subject to change later on
<pre>private void levelOne() { //floor addObject(new Platform(900, 40), 450, 580); }</pre>	This is the code for the floor of the levels. The platform has parameters 900, 40 which determine the width and height
<pre>import greenfoot.*; public class Platform extends Actor { public Platform(int width, int height) { //load the platform image GreenfootImage image = new GreenfootImage("platform.png"); //resize it to the size given in GameWorld image.scale(width, height); setImage(image); } }</pre>	The image is loaded by the constructor, then it is resized by the next line of code where it is changed to the size of the

Test 2:

Purpose: to check that my game loads the floor correctly

Type of test: normal

Test data: run program

Expected outcome: my game opens without errors and the floor is at the bottom of the screen, covering the entire width

Actual outcome: unsuccessful

The game would run but I saw that my platform was skewed to the left, so I decided to tweak my method to fix this



Part 2 revised:

<pre>private void levelOne() { //floor addObject(new Platform(900, 40), 450, 580); }</pre> Changed to: <pre>private void levelOne() { //floor addObject(new Platform(screenWidth, 40), screenWidth/2, 580); }</pre>	The first change makes it so my platform always covers the entire width of my game screen, the second change makes it so the floor is always centred on the screen
--	---

Purpose: to recheck that my game loads the floor correctly

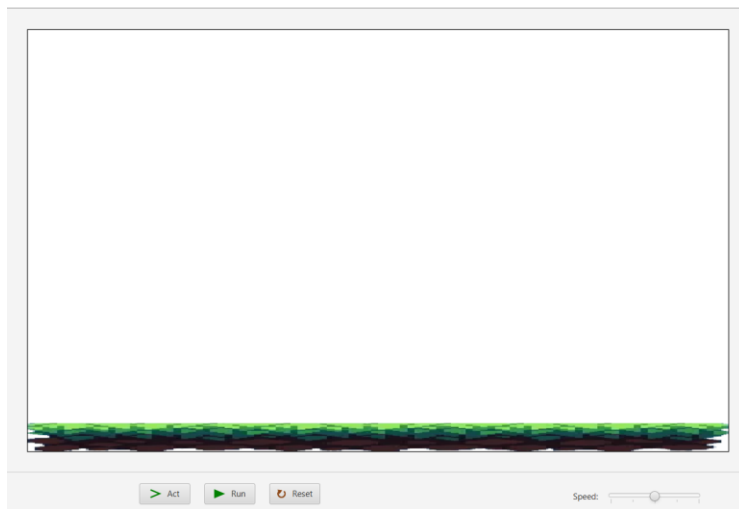
Type of test: normal

Test data: run program

Expected outcome: my game opens without errors and the floor is at the bottom of the screen, covering the entire width

Actual outcome: successful

The game would run and my platform was the correct size and in the correct place



After working with my code I decided to put some of the key variables/functions into a separate class called GameSettings. This will help with the codes readability and make it easier to work with as those key elements are separated

Within the GameSettings class I added the necessary variables that change how the game plays regarding the difficulty level

	When this method is called it changes the variables shown to the respective values shown (it changes the game between hard and easy mode)
<pre>public static String difficulty = "Easy"; public static int levelNumber = 1; public static int startingHearts = 5; public static int enemySpeed = 2;</pre>	These are the variables used and are pre-set to easy mode so it is easier for a student to understand how the variables might be used